National University of Singapore

CS1101S — Programming Methodology

AY2022/2023 Semester 2

Final Assessment

Time allowed: 2 hours

QUESTION PAPER

INSTRUCTIONS

1. This **QUESTION PAPER** contains **21 Questions** in **5 Sections**, and comprises **11** printed pages, including this page.
2. The **ANSWER SHEET** comprises **10** printed pages.
3. Use a pen or pencil to **write** your **Student Number** in the designated space on the front page of the **ANSWER SHEET**, and **shade** the corresponding circle **completely** in the grid for each digit or letter. DO NOT WRITE YOUR NAME!
4. You must **submit only** the **ANSWER SHEET** and no other documents. Do not tear off any pages from the ANSWER SHEET.
5. All questions must be answered in the space provided in the **ANSWER SHEET**; no extra sheets will be accepted as answers.
6. Write legibly with a **pen** or **pencil** (do not use red color). Untidiness will be penalized.
7. For **multiple choice questions (MCQ)**, **shade** in the **circle** of the correct answer **completely**.
8. The full score of this assessment is **100 marks**.
9. This is a **Closed-Book** assessment, but you are allowed to bring with you one double-sided **A4 / letter-sized sheet** of handwritten or printed **notes**.
10. Where programs are required, write them in the **Source §3** language. A **reference** of some of the **pre-declared functions** is given in the **Appendix** of the Question Paper.
11. In any question, unless it is specifically allowed, your answer **must not use functions** given in, or written by you for, other questions.

Section A: Pair Structures [20 marks]

For each of the following Source §3 programs, draw the box-and-pointer diagram for the value of `result` at the end of the program evaluation. Clearly show where `result` is pointing to. For pairs that are *identical* (===), only one box must be shown. Pairs that are *not identical* must be drawn as separate boxes.

(1) [4 marks]

```
const result = list(list(2), 3, 4);
set_tail(head(result), tail(result));
```

(2) [4 marks]

```
const result = pair(null, list(list(2), 3));
set_head(result, map(x => x, tail(result)));
```

(3) [4 marks]

```
let xx = pair(1, null);
let yy = pair(2, xx);
let result = pair(3 , yy);
set_head(yy, result);
set_head(xx, yy);
```

(4) [4 marks]

```
function join(xs, ys) {
    if (is_null(xs)) {
        return ys;
    } else {
        set_tail(xs, join(tail(xs), ys));
        return xs;
    }
}
const yy = list(2, 4, 6);
const result = join(yy, yy);
```

(5) [4 marks]

```
function binop(x, y) {
    const p = pair(tail(y), x);
    set_tail(y, p);
    return y;
}
const result = accumulate(binop, pair(0, null), list(1, 2, 3, 4));
```

Section B: Orders of Growth [18 marks]

What is the **order of growth** of the **running time** of each of the following functions in terms of N using the Θ notation? Note that N is a positive integer value.

(6) [3 marks] (MCQ)

```
function fun(N) {
    let acc = 1;
    for (let i = 1; i <= N / 2; i = i + 2) { acc = acc * 2; }
    return acc;
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

(7) [3 marks] (MCQ)

```
function fun(N) {
    let acc = 1;
    for (let i = 1; i <= N; i = 10 * i) { acc = acc + 10; }
    return acc;
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

(8) [3 marks] (MCQ)

```
function fun(N) {
    let acc = 1;
    for (let i = 1; i <= N; i = i + 2) {
        for (let k = i; k >= 1; k = k - 2) { acc = acc + 2; }
    }
    return acc;
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

(9) [3 marks] (MCQ)

```
function fun(N) {
    let acc = 1;
    for (let i = 1; i <= 3 * N; i = i + 3) { acc = acc + 3; }
    for (let j = 1; j <= 5 * N; j = j + 5) { acc = acc + 5; }
    return acc;
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

(10) [3 marks] (MCQ)

```
function fun(N) {
    let acc = 1;
    for (let i = 1; i <= N * N * N; i = i + N) { acc = acc + 1; }
    return acc;
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

(11) [3 marks] (MCQ)

```
function fun(N) {
    let acc = 1;
    for (let i = N; i >= 1; i = i / 2) {
        for (let j = 1; j <= i ; j = j + 1) {
            for (let k = 1; k <= N / i; k = k + 1) { acc = acc + 1; }
        }
    }
    return acc;
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

Section C: Program Evaluation [20 marks]

For each of the following programs, **write the sequence of values printed** by the `display` function when the program is evaluated. The `display` function displays/prints the value of its input argument in the REPL, *and* returns the value of its input argument. For example, `display(2 * 5)` prints `10` and returns `10`; and `display(1 > 2)` prints `false` and returns `false`.

(12) [5 marks]

```
function D(x, y) {
    display(stringify(x) + ", " + stringify(y));
}

function fun(N) {
    for (let i = 1; i <= N; i = i + 1) {
        for (let j = N; j >= i; j = j - 2) {
            D(i, j);
        }
    }
}

fun(5);
```

(13) [5 marks]

```
const D = display;

function fun(x, y) {
    return D( D(5 + x) * D(4 + y) ); // (5 + x) * (4 + y)
}

fun( D(3), D(2) ); // equivalent to fun(3, 2);
```

(14) [5 marks]

```
function D(m, x) {
    display(m);
    return x;
}
const w = D(5, 5);
const z = D( "fn", x => D("+", D("x", x) + D("w", w)) ) ( D(20, 20) );
    // equivalent to (x => x + w)(20)
D(z, z);
```

(15) [5 marks]

```
function binary_search(A, v) {
    function search(low, high) {
        if (low > high) {
            return false;
        } else {
            const mid = math_floor((low + high) / 2);
            display(mid); // display is here
            return (v === A[mid]) ||
                   (v < A[mid]
                    ? search(low, mid - 1)
                    : search(mid + 1, high));
        }
    }
    return search(0, array_length(A) - 1);
}
const AA = [0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19];
    // array_length(AA) is 20
binary_search(AA, 16);
```

Section D: Environment Model [16 marks]

For each of the following Source programs, draw the diagram to show the environment during the evaluation of the program. Show all the frames that are created during the program evaluation (except the global environment frame), and show the final value of each binding.

Note that the application of a **primitive** function (e.g. `array_length`, `pair`, `display`, `tail`) does not create a new environment frame.

(16) [8 marks]

Program D1:

```
let z = 4;

function g() {
    let z = 5;
    const h = x => (y => 100 * x + 10 * y + z);
    z = 6;
    return h;
}

z = 7;
const h = g()(2)(3);
```

(17) [8 marks]

Program D2:

```
function foo(n) {
    const A = [];
    let i = 0;
    while (i < n) {
        const f = () => i;
        A[i] = f;
        i = i + 1;
    }
    return A;
}

function bar(H, i, n) {
    return i === n ? 0 : H[i]() + bar(H, i + 1, n);
}

const N = 2;
const F = foo(N);
const X = bar(F, 0, N);
```

Section E: List Processing [26 marks]

(18) [7 marks]

Complete the declaration of function `sum_every_pair` that takes as argument a list of numbers `xs`, and returns a list of numbers whose each element is the sum of two consecutive elements of `xs`. If the length of `xs` is odd, then the last element of the result list will just be the last element of the input list `xs`. Your function must not modify the input list `xs`.

The running time of your function must have an order of growth of $O(n)$, where n is the length of the input list `xs`.

Examples:

```
sum_every_pair(null);
// return null

sum_every_pair(list(1, 2, 3, 4, 5, 6));
// return list(3, 7, 11)

sum_every_pair(list(1, 2, 3, 4, 5));
// return list(3, 7, 5)
```

(19) [7 marks]

Complete the declaration of function `d_sum_every_pair` that takes as argument a list of numbers `xs`, and returns a list of numbers whose each element is the sum of two consecutive elements of `xs`. If the length of `xs` is odd, then the last element of the result list will just be the last element of the input list `xs`. Your function **must not create any new pair**, and the result list must be made of existing pairs in `xs`.

The running time of your function must have an order of growth of $O(n)$, where n is the length of the input list `xs`.

Examples:

```
d_sum_every_pair(null);
// return null

d_sum_every_pair(list(1, 2, 3, 4, 5, 6));
// return list(3, 7, 11)

d_sum_every_pair(list(1, 2, 3, 4, 5));
// return list(3, 7, 5)
```

Write your answer only in the dashed-line box(es).

(20) [8 marks]

Complete the declaration of function `d_insert` that takes as arguments, `p`, a pair whose head is a number, and, `xs`, a sorted list of numbers (in ascending order), and returns a sorted list of numbers by inserting `p` into `xs`. Your function **must not create any new pair**, and the result list must be made of existing pair `p` and pairs in `xs`. Your function must not modify the head of any of the existing pairs (i.e. must not use the `set_head` function).

The running time of your function must have an order of growth of $O(n)$, where n is the length of the input list `xs`.

Examples:

```
d_insert(list(3), list(1, 2, 4, 5, 6));
// returns list(1, 2, 3, 4, 5, 6)

d_insert(list(8), null);
// returns list(8)
```

Write your answer only in the dashed-line box(es).

(21) [4 marks]

Complete the declaration of function `d_insertion_sort` that takes as argument a list of numbers `xs`, and returns a sorted list (in ascending order) of the elements of the input list by performing insertion sort. Your function **must not create any new pair**, and the result list must be made of existing pairs in `xs`. Your function must not modify the head of any of the existing pairs (i.e. must not use the `set_head` function).

Your function must make use of the `d_insert` function from the preceding question, and it is assumed to be correctly implemented.

The running time of your function must have an order of growth of $O(n^2)$, where n is the length of the input list `xs`.

Example:

```
d_insertion_sort(list(5, 2, 3, 6, 1, 4));
// returns list(1, 2, 3, 4, 5, 6)
```

——— END OF QUESTIONS ———

Appendix

We assume the following pre-declared functions in Source §3 are declared as follows:

```
function map(f, xs) {
    return is_null(xs)
           ? xs
           : pair(f(head(xs)), map(f, tail(xs)));
}

function filter(pred, xs) {
    return is_null(xs)
           ? null
           : pred(head(xs))
           ? pair(head(xs), filter(pred, tail(xs)))
           : filter(pred, tail(xs));
}

function accumulate(f, initial, xs) {
    return is_null(xs)
           ? initial
           : f(head(xs), accumulate(f, initial, tail(xs)));
}

function append(xs, ys) {
    return is_null(xs)
           ? ys
           : pair(head(xs), append(tail(xs), ys));
}
```

The following **LISTS** functions are supported in Source §3:

- `pair(x, y)`: Makes a pair from `x` and `y`.
- `is_pair(x)`: Returns `true` if `x` is a pair and `false` otherwise.
- `head(x)`: Returns the head (first component) of the pair `x`.
- `tail(x)`: Returns the tail (second component) of the pair `x`.
- `set_head(p, x)`: Changes the pair `p` such that its head is `x`.
- `set_tail(p, x)`: Changes the pair `p` such that its tail is `x`.
- `is_null(xs)`: Returns `true` if `xs` is the empty list, and `false` otherwise.
- `is_list(x)`: Returns `true` if `x` is a list as defined in the lectures, and `false` otherwise. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of the chain of `tail` operations that can be applied to `x`.
- `list(x1, x2,..., xn)`: Returns a list with `n` elements. The first element is `x1`, the second `x2`, etc. Iterative process; time: $O(n)$, space: $O(n)$, since the constructed list data structure consists of n pairs, each of which takes up a constant amount of space.
- `length(xs)`: Returns the length of the list `xs`. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `map(f, xs)`: Returns a list that results from list `xs` by element-wise application of `f`. **Recursive process**; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `build_list(f, n)`: Makes a list with `n` elements by applying the unary function `f` to the numbers 0 to `n - 1`. Iterative process; time: $O(n)$, space: $O(n)$.

- `for_each(f, xs)`: Applies `f` to every element of the list `xs`, and then returns `true`. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `reverse(xs)`: Returns list `xs` in reverse order. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`. The process is iterative, but consumes space $O(n)$ because of the result list.
- `append(xs, ys)`: Returns a list that results from appending the list `ys` to the list `xs`. **Recursive process**; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `member(x, xs)`: Returns first postfix sublist whose head is identical to `x` (`===`); returns `null` if the element does not occur in the list. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `remove(x, xs)`: Returns a list that results from `xs` by removing the first item from `xs` that is identical (`===`) to `x`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `remove_all(x, xs)`: Returns a list that results from `xs` by removing all items from `xs` that are identical (`===`) to `x`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `filter(pred, xs)`: Returns a list that contains only those elements for which the one argument function `pred` returns `true`. **Recursive process**; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `enum_list(start, end)`: Returns a list that enumerates numbers starting from `start` using a step size of 1, until the number exceeds (`>`) `end`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`. For example, `enum_list(2, 5)` returns the list `list(2, 3, 4, 5)`.
- `list_ref(xs, n)`: Returns the element of list `xs` at position `n`, where the first element has index 0. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `accumulate(op, initial, xs)`: Applies binary function `op` to the elements of `xs` from right-to-left order, first applying `op` to the last element and the value `initial`, resulting in r_1, then to the second-last element and r_1, resulting in r_2, etc, and finally to the first element and r_{n-1}, where n is the length of the list. Thus, `accumulate(op, zero, list(1,2,3))` results in `op(1, op(2, op(3, zero)))`. **Recursive process**; time: $O(n)$, space: $O(n)$, where n is the length of `xs`, assuming `op` takes constant time.

Some other functions supported in Source §3:

- `is_boolean(x)`: Returns `true` if `x` is a boolean value, and `false` otherwise.
- `is_number(x)`: Returns `true` if `x` is a number, and `false` otherwise.
- `is_string(x)`: Returns `true` if `x` is a string, and `false` otherwise.
- `is_array(x)`: Returns `true` if `x` is a boolean value, and `false` otherwise.
- `array_length(x)`: Returns the current length of array `x`.
- `display(v)`: Displays the value `v` in the REPL.
- `stringify(v)`: Returns a string that represents the value `v`. For example, `stringify(123)` returns the string `"123"`, and `stringify(false)` returns the string `"false"`.
- `math_floor(x)`: Returns the largest integer that is equal to or less than the number `x`.
- `math_sqrt(x)`: Returns the square root of the number `x`.
- `math_pow(base, exp)`: Returns the result of raising `base` to the power of `exp`.
- `math_min(x1, x2,..., xn)`: Returns the smallest of the `n` arguments. Time: $O(n)$, space: $O(1)$.
- `math_max(x1, x2,..., xn)`: Returns the largest of the `n` arguments. Time: $O(n)$, space: $O(1)$.

——— END OF PAPER ———

National University of Singapore

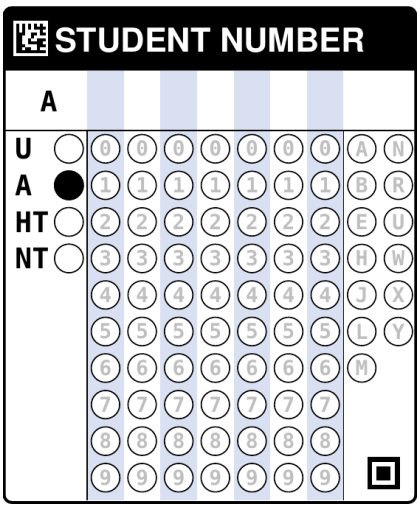
CS1101S — Programming Methodology

AY2022/2023 Semester 2

Final Assessment

Time allowed: 2 hours

ANSWER SHEET



(1) [4 marks]

(2) [4 marks]

(3) [4 marks]

(4) [4 marks]

(5) [4 marks]

(6) [3 marks] (MCQ)

◯ **A** ◯ **B** ◯ **C** ◯ **D** ◯ **E** ◯ **F** ◯ **G** ◯ **H**

(7) [3 marks] (MCQ)

◯ **A** ◯ **B** ◯ **C** ◯ **D** ◯ **E** ◯ **F** ◯ **G** ◯ **H**

(8) [3 marks] (MCQ)

◯ **A** ◯ **B** ◯ **C** ◯ **D** ◯ **E** ◯ **F** ◯ **G** ◯ **H**

(9) [3 marks] (MCQ)

◯ **A** ◯ **B** ◯ **C** ◯ **D** ◯ **E** ◯ **F** ◯ **G** ◯ **H**

(10) [3 marks] (MCQ)

◯ **A** ◯ **B** ◯ **C** ◯ **D** ◯ **E** ◯ **F** ◯ **G** ◯ **H**

(11) [3 marks] (MCQ)

◯ **A** ◯ **B** ◯ **C** ◯ **D** ◯ **E** ◯ **F** ◯ **G** ◯ **H**

(12) [5 marks]

(13) [5 marks]

(14) [5 marks]

(15) [5 marks]

(16) [8 marks]

Environment of Program D1:

(17) [8 marks]

Environment of Program D2:

(18) [7 marks]

```
function sum_every_pair(xs) {




}
```

(19) [7 marks]

```
function d_sum_every_pair(xs) {
    if (is_null(xs)) {
       ┌──────────────────────────────────────────────────────────┐
       │                                                          │
       └──────────────────────────────────────────────────────────┘
    } else if (is_null(tail(xs))) {
       ┌──────────────────────────────────────────────────────────┐
       │                                                          │
       └──────────────────────────────────────────────────────────┘
    } else {
       ┌──────────────────────────────────────────────────────────┐
       │                                                          │
       │                                                          │
       │                                                          │
       │                                                          │
       │                                                          │
       └──────────────────────────────────────────────────────────┘
    }
}
```

(20) [8 marks]

```
function d_insert(p, xs) {
    if (is_null(xs)) {
        
    } else if (head(p) <= head(xs)) {
        
    } else {
        
    }
}
```

(21) [4 marks]

```
function d_insertion_sort(xs) {




}
```

——— END OF ANSWER SHEET ———